

BÀI 28: PHẠM VI CỦA BIẾN

SAU BÀI NÀY EM SẼ

- Biết và trình bày được ý nghĩa của phạm vi hoạt động của biến trong chương trình và trong hàm.

KHỞI ĐỘNG

1. Một biến được định nghĩa trong chương trình chính (bên ngoài các hàm) thì sẽ được sử dụng như thế nào bên trong các hàm?
2. Một biến được khai báo bên trong một hàm thì có sử dụng được ở bên ngoài hàm đó hay không?

Bài này sẽ giúp em tìm câu trả lời cho các câu hỏi trên.

1. PHẠM VI CỦA BIẾN KHAI BÁO TRONG HÀM (BIẾN CỤC BỘ)

Hoạt động 1: Phạm vi của biến khi khai báo trong hàm

Quan sát các lệnh sau để tìm hiểu phạm vi của biến khi khai báo bên trong một hàm.

Python

Định nghĩa hàm

```
>>> def func(a, b):
```

```
...     n = 10      # 'n' là biến cục bộ của hàm func
```

```
...     a = a * 2    # 'a' và 'b' cũng là biến cục bộ trong hàm này
```

```
...     b = a + b
```

```
...     return a + b + n
```

```
...
```

Chương trình chính

```
>>> a = 1          # 'a' và 'b' ở đây là biến toàn cục
```

```
>>> b = 2
```

```
>>> func(a, b)     # Gọi hàm, kết quả trả về là 16
```

```
16
```

```
>>> print(a, b)    # Giá trị của a, b toàn cục không thay đổi
```

```
(1, 2)
```

```
>>> print(n)       # Cố gắng truy cập biến cục bộ 'n' từ bên ngoài
```

Traceback (most recent call last):

```
...
```

NameError: name 'n' is not defined

Giải thích:

- Các biến được khai báo bên trong một hàm (n, và các tham số a, b của hàm) được gọi là **biến cục bộ (local variable)**.
- Chúng chỉ tồn tại và có hiệu lực **bên trong hàm đó**. Mọi thay đổi với chúng không ảnh hưởng đến các biến bên ngoài, kể cả các biến có cùng tên.
- Khi hàm kết thúc, các biến cục bộ sẽ bị hủy. Do đó, ta không thể truy cập chúng từ bên ngoài hàm, dẫn đến lỗi NameError.

GHI NHỚ

Trong Python, tất cả các biến được khai báo bên trong hàm đều có tính địa phương (cục bộ), không có hiệu lực ở bên ngoài hàm.

2. PHẠM VI CỦA BIẾN KHAI BÁO NGOÀI HÀM (BIẾN TOÀN CỤC)

Hoạt động 2: Phạm vi của biến khi khai báo bên ngoài hàm

Biến được khai báo bên ngoài tất cả các hàm được gọi là biến toàn cục (global variable).

Ví dụ 1: Truy cập (đọc) biến toàn cục.

Bên trong một hàm, ta có thể đọc giá trị của một biến toàn cục.

Python

```
>>> def f(a, b):  
...     return a + b + N # Hàm f() đọc giá trị của biến toàn cục N  
...  
>>> N = 10          # N là biến toàn cục  
>>> f(1, 2)  
13
```

Ví dụ 2: Thay đổi biến toàn cục từ bên trong hàm.

Mặc định, ta không thể thay đổi giá trị của một biến toàn cục từ bên trong hàm. Nếu ta thực hiện phép gán, Python sẽ tạo ra một biến cục bộ mới có cùng tên.

Python

```
>>> def f(n):  
...     t = 2 * n + 1 # Python tạo ra biến cục bộ 't' mới  
...     return t  
...  
>>> t = 10          # 't' là biến toàn cục  
>>> f(1)            # Hàm trả về 3, nhưng không thay đổi t toàn cục  
3  
>>> print(t)        # 't' toàn cục vẫn là 10  
10
```

Lưu ý: Từ khoá global

Nếu muốn thay đổi giá trị của một biến toàn cục từ bên trong hàm, ta phải khai báo biến đó với từ khoá global.

Python

```
>>> def f(n):  
...     global t      # Khai báo rằng ta muốn dùng biến toàn cục 't'  
...     t = 2 * n + 1 # Bây giờ lệnh này sẽ thay đổi 't' toàn cục  
...     return t  
...  
>>> t = 10  
>>> f(1)  
3  
>>> print(t)        # Giá trị của 't' toàn cục đã bị thay đổi  
3
```

GHI NHỚ

- Biến khai báo bên ngoài hàm là biến **toàn cục**.
- Bên trong hàm có thể **đọc** giá trị của biến toàn cục.
- Để **thay đổi** giá trị của biến toàn cục từ bên trong hàm, cần khai báo lại biến đó trong hàm với từ khoá `global`.

THỰC HÀNH

Nhiệm vụ 1: Viết hàm với đầu vào là danh sách A và số thực x. Hàm trả lại một danh sách kết quả B chỉ chứa các phần tử của A lớn hơn hoặc bằng x.

Python

```
def Select(A, x):
    B = [] # B là biến cục bộ, được tạo mới mỗi khi gọi hàm
    for k in A:
        if k >= x:
            B.append(k)
    return B
```

Nhiệm vụ 2: Viết chương trình gồm nhiều hàm để:

1. **Hàm 1:** Nhập một dãy số nguyên từ bàn phím và lưu vào danh sách A.
2. **Hàm 2:** Trích từ A ra danh sách B gồm các phần tử lớn hơn 0.
3. **Hàm 3:** Trích từ A ra danh sách C gồm các phần tử nhỏ hơn 0.

Python

```
def Nhap_Dulieu():
    s = input("Nhập các số nguyên cách nhau bởi dấu cách: ")
    A_str = s.split()
    A = []
    for k in A_str:
        A.append(int(k))
    return A
```

```
def getB(A):
    B = []
    for x in A:
        if x > 0:
            B.append(x)
    return B
```

```
def getC(A):
    C = []
    for x in A:
        if x < 0:
            C.append(x)
    return C
```

--- Chương trình chính ---

```
A = Nhap_Dulieu()
print("Danh sách A:", A)
B = getB(A)
print("Danh sách B (số dương):", B)
C = getC(A)
print("Danh sách C (số âm):", C)
```

LUYỆN TẬP

1. Viết hàm `Tach_day(A)` với đầu vào là danh sách A, đầu ra là hai danh sách B (gồm các phần tử có chỉ số chẵn của A) và C (gồm các phần tử có chỉ số lẻ của A).

2. Viết hàm `get_initials(sList)` với đầu vào là danh sách các chuỗi `sList`, đầu ra là danh sách `cList` chứa các kí tự đầu tiên của các chuỗi tương ứng trong `sList`.

VẬN DỤNG

1. Viết hàm có hai tham số đầu vào là `m`, `n`. Hàm trả lại hai giá trị là $UCLN(m, n)$ và $BCNN(m, n)$.
 - **Gợi ý:** Sử dụng công thức $UCLN(m,n) \times BCNN(m,n) = m \times n$.
2. Viết chương trình nhập ba số tự nhiên `day`, `month`, `year` từ bàn phím (cách nhau bởi dấu cách). Chương trình cần kiểm tra và in ra thông báo xem ngày tháng năm đó có hợp lệ hay không.